RAG Pipeline - Advanced Indexing: Multi-Representation

This document covers advanced indexing techniques that go beyond the simple "chunk and embed" model.

1. Definition

Multi-Representation Indexing (or Multi-Vector Indexing) is a family of advanced techniques based on one core idea: a single document should be "represented" by *more than one vector* in your database.

The goal is to solve the **Query-Document Mismatch Problem**. A user's query (e.g., a short question) is often semantically very different from a long, dense document chunk (a long answer). By creating *alternative, concise representations* (like summaries or hypothetical questions) and embedding those instead, you create vectors that are much "closer" to the user's query vector, leading to far more accurate retrieval.

2. Core Problem: The Query-Document Mismatch

- **Query:** "What is the policy on sick leave for part-time workers?" (Short, specific, a question)
- **Document:** "Section 9.b of the employee handbook states: 'Full-time employees (FTEs) are granted 40 hours of sick leave per fiscal year... Our company values a healthy work-life balance for all staff, including part-time and contract workers. For non-FTE staff, sick leave is accrued at a rate of 1 hour for every 30 hours worked...'" (Long, dense, an answer, mixed with other info)

A simple vector search might not find this document because the query and the document "look" very different in the vector space.

3. Key Techniques & LangChain Tools

A. Parent Document Retriever (Small-to-Large)

This is one of the most popular and effective techniques. It gives you the best of both worlds: the precision of small chunks and the context of large documents.

- **What is it?** You split your documents twice: once into large "parent" chunks and again into small "child" chunks. You only embed the *small, child chunks* for searching. When one is found, you retrieve the *large, parent chunk* it belongs to.
- **How it works (Process):**
    1. **Index Time:**
        - Split docs into large (e.g., 2000-character) "parent" documents.
        - Split those parent docs into smaller (e.g., 400-character) "child" documents.
        - Store the parent docs in a key-value `docstore`.

- Embed and store *only* the child docs in the `vectorstore`, with a pointer (metadata) to their parent's ID.

2. **Retrieval Time:**
   - The user's query is embedded.
   - It finds the *small, child chunks* that are the best match (high precision).
   - The retriever uses the pointers from those child chunks to retrieve the *full parent documents* from the `docstore`.
   - These large, context-rich parent documents are passed to the LLM.

- **Key LangChain Tool:** `ParentDocumentRetriever`. This tool automates the entire process (splitting, storing in both docstore and vectorstore, and the two-step retrieval).

B. Summary-Based Retriever (Your Definition)

This is the exact technique you described. It's excellent for long, complex documents.

- **What is it?** You generate a concise summary for each document (or large chunk). You embed the *summary* for retrieval but retrieve the *original document*.
- **How it works (Process):**
  1. **Index Time:**
     - For each document, use an LLM to generate a summary.
     - Store the original document in a `docstore`.
     - Embed and store the *summary* in the `vectorstore`, with a pointer to the original document's ID.
  2. **Retrieval Time:**
     - The user's query (e.g., "What are the main arguments in this paper?") is embedded.
     - It finds the *summary* that is the best match (high similarity).
     - The retriever uses the pointer to fetch the *original, full document* from the `docstore`.
     - This full document is passed to the LLM.
- **Key LangChain Tool:** `MultiVectorRetriever`. This is the flexible, underlying engine that lets you build this pattern. You provide the logic for creating the summaries and tell it how to link the vectors (summaries) to the documents (originals).

C. Hypothetical Question Retriever

This technique is the inverse of HyDE (which we discussed in Query Translation). Instead of generating a fake *answer* at query time, you generate fake *questions* at index time.

- **What is it?** For each document, you generate 3-5 hypothetical questions that the document *could answer*. You embed these *questions* instead of the document itself.
- **How it works (Process):**

1. **Index Time:**
    - For each document, use an LLM to generate a list of questions (e.g., "What is the policy for part-time sick leave?").
    - Store the original document in a `docstore`.
    - Embed and store these *hypothetical questions* in the `vectorstore`, all pointing to the *same* original document ID.
2. **Retrieval Time:**
    - The user's *real query* ("How much sick leave do part-timers get?") is embedded.
    - It finds the *hypothetical question* that is the best match (which will be very high, as they are both questions).
    - The retriever uses the pointer to fetch the *original, full document* from the `docstore`.
    - This full document is passed to the LLM.

- **Key LangChain Tool:** `MultiVectorRetriever`. This is another pattern built using this flexible retriever.